

INSTITUIÇÃO: UNIFACS — Universidade Salvador	PROJETO: Gêmeo Digital IoT Pelourinho
PROGRAMA: Iniciação Científica & Inovação Tecnológica	DATA: 18 de Setembro de 2026
SUBSISTEMA: Doc 01 — Macro-Fluxo Circadiano (\$w_j\$)	STATUS: Código Implementado & Homologado

Relatório Técnico: Implementação, Código & Validação

Resolução do Doc 01 — Macro-Fluxo Circadiano & Distribuição por Portões de Entrada

1. Sumário Executivo & Contexto

O Centro Histórico do Pelourinho comporta-se como um **sistema urbano aberto**: turistas e residentes acessam o bairro por portões geográficos principais (Elevador Lacerda, Praça da Sé, Ladeira do Carmo) ao longo do dia e dispersam ao anoitecer. O **Doc 01** define a modelagem estocástica basal desse fluxo contínuo, servindo de fundação para toda a suíte de simulação do Gêmeo Digital IoT.

Destaques da Solução Implementada:

- Código-Fonte de Referência:** Pacote `src.simulation` construído com motor matemático, schemas e CLI;
- Conformidade Fiel:** Equação analítica fechada do Sino Gaussiano calibrada para ápice às 16h30 (300 pessoas);
- Conservação Física de Fluxo:** Restrição estrita $\sum w_j = 1.0$ validada no nível de modelo e de runtime;
- Desempenho Extremo:** Cálculo em **1.356 μ s** por ciclo (complexidade $O(1)$, abaixo do teto de 10 μ s);
- Homologação Completa:** 10 testes unitários cobrindo RF01-RF05 e RNF01-RNF03 aprovados sem falhas.

2. Formulação Matemática & Soluções de Engenharia

A lotação global do Centro Histórico $N_{\text{bairro}}(t)$ é modelada analiticamente pela função fechada:

$$N_{\text{bairro}}(t) = \gamma \cdot [N_{\text{min}} + (N_{\text{max}} - N_{\text{min}}) \cdot \exp(-(h(t) - t_{\text{pico}})^2 / (2\sigma^2))]$$

Onde: $h(t) = t \bmod 24$ | $N_{\text{min}} = 15$ | $N_{\text{max}} = 300$ | $t_{\text{pico}} = 16.5$ (16h30) | $\sigma = 3.0h$ | $\gamma = \text{fator sazonal}$

A alocação espacial para cada nó j é dada por: $N_{\text{rotina},j}(t) = w_j \cdot N_{\text{bairro}}(t)$, com conservação $\sum w_j = 1.0$ (para $j = 1 \dots J$).

Portão de Acesso / Nó	Sensor ID	Peso w_j	Lotação Máx	Função Urbana
Elevador Lacerda	sensor_elevador_lacerda	0.45 (45%)	150 pessoas	Portão Principal
Praça da Sé	sensor_praca_da_se	0.35 (35%)	120 pessoas	Acesso Sul
Ladeira do Carmo	sensor_ladeira_do_carmo	0.20 (20%)	80 pessoas	Acesso Norte
Largo do Pelourinho	sensor_largo_pelourinho	0.00 (0%)	200 pessoas	POI Interno (sem entrada)
TOTAL CONSERVADO	4 sensores	1.00 (100%)	550 pessoas	Restrição RF03 Satisfeita

3. Mapeamento das Mudanças no Repositório

Arquivo / Localização	Tipo	Descrição da Mudança / Funcionalidade
backend/src/simulation/__init__.py	Novo	Inicializador do pacote modular de simulação; expõe classes e schemas.
backend/src/simulation/schemas.py	Novo	Modelos Pydantic v2 com validação RF03 e fallback resiliente para dataclasses.
backend/src/simulation/macro_flow.py	Novo	Motor matemático analítico contínuo $O(1)$, suporte NumPy e geração de curvas.
backend/src/simulation/cli_macro_flow.py	Novo	CLI interativa com UTF-8 para testes manuais e inspeção via terminal.
backend/src/api/routers/simulation.py	Novo	Endpoints REST (/macroflow, /curve, /default-config) na API FastAPI.
backend/src/api/main.py	Modificado	Registro oficial do router de simulação sob os prefixos /api/v1 e /api.
backend/tests/test_macro_flow.py	Novo	Suíte formal de 10 testes unitários validando RFs, RNFs e benchmarks.

.agents/memory/progress.md	Modificado	Atualização da memória viva registrando a conclusão com sucesso do Doc 01.
----------------------------	------------	--

4. Código-Fonte Utilizado na Resolução do Problema

Abaixo são apresentados os trechos-chave do código em Python implementados para materializar a formulação matemática do Doc 01, organizados por responsabilidade técnica:

Listagem 4.1: Motor Analítico e Relógio Circadiano (backend/src/simulation/macro_flow.py)

```
# backend/src/simulation/macro_flow.py – Núcleo Matemático em O(1)
def circadian_hour(t_hours: float) -> float:
    """Calcula a hora circadiana continua  $h(t) = t \bmod 24$  no intervalo  $[0, 24)$ ."""
    return float(t_hours % 24.0)

def calculate_bairro_volume(
    t_hours: float, gamma: float = 1.0,
    N_min: float = 15.0, N_max: float = 300.0,
    t_peak: float = 16.5, sigma: float = 3.0,
) -> float:
    """Avalia a exata formula analitica fechada do Sino Gaussiano (RF01, RF04)."""
    h = circadian_hour(t_hours)
    diff = h - t_peak
    exponent = -(diff * diff) / (2.0 * sigma * sigma)
    bell = 0.0 if exponent < -50.0 else math.exp(exponent)
    return float(max(0.0, gamma * (N_min + (N_max - N_min) * bell)))

def allocate_gates(N_bairro: float, weights: Sequence[float]) -> Union[List[float], np.ndarray]:
    """Aloca N_bairro(t) nos portoes via produto vetorial ponderado (RF02)."""
    if HAS_NUMPY:
        return np.asarray(weights, dtype=np.float64) * N_bairro
    return [float(w * N_bairro) for w in weights]
```

Listagem 4.2: Validador de Conservação de Fluxo (backend/src/simulation/schemas.py)

```
# backend/src/simulation/schemas.py – Validacao da Conservacao de Fluxo (RF03)
class MacroFlowConfig(BaseModel):
    N_max_bairro: int = Field(300, gt=0)
    N_min_bairro: int = Field(15, ge=0)
    t_peak: float = Field(16.5, ge=0.0, lt=24.0)
    sigma: float = Field(3.0, gt=0.0)
    gates: List[SensorGateWeight] = Field(...)

    @field_validator("gates")
    @classmethod
    def validate_weights_sum(cls, gates: List[SensorGateWeight]) -> List[SensorGateWeight]:
        """Garante a conservacao de fluxo:  $\sum(w_j) == 1.0$  (tolerancia 1e-4)."""
        total = sum(g.gate_weight for g in gates)
        if abs(total - 1.0) > 1e-4:
            raise ValueError(f"Violacao de RF03: soma dos pesos = {total:.4f} != 1.0")
        return gates
```

Listagem 4.3: Exposição REST na API FastAPI (backend/src/api/routers/simulation.py)

```
# backend/src/api/routers/simulation.py – Endpoints REST FastAPI
@router.post("/macroflow", response_model=MacroFlowResponse)
def evaluate_macro_flow(payload: MacroFlowRequest):
    """Calcula o volume global e aloca aos portoes de entrada (Doc 01)."""
    sim = _DEFAULT_SIMULATOR
    if payload.config_override:
        sim = MacroFlowSimulator(config=payload.config_override)
    result = sim.evaluate(t_hours=payload.current_time_hours, gamma=payload.gamma_seasonality)
    return result.to_response()

@router.get("/macroflow/curve", response_model=MacroFlowCurveResponse)
def get_macro_flow_24h_curve(step_minutes: float = 15.0, gamma: float = 1.0):
    """Retorna a serie temporal completa de 24 horas para o mapa Leaflet."""
    curve_data = _DEFAULT_SIMULATOR.generate_24h_curve(step_hours=step_minutes/60.0, gamma=gamma)
    return MacroFlowCurveResponse(curve=[MacroFlowCurvePoint(**p) for p in curve_data], ...)
```

5. Resultados Experimentais & Homologação Formal

A validação foi conduzida via unittest através de 10 testes formais automatizados. A suíte completa executou em apenas **0.011 segundos** com 100% de taxa de sucesso.

Teste Unitário	Requisito Alvo	Critério de Sucesso	Status
test_exact_gaussian_peak	RF01	Às 16h30 (t=16.5), volume global N_bairro = 300.0	PASS
test_circadian_midnight_baseline	RF01	Às 03h00, volume converge assintoticamente para 15.0	PASS
test_conservation_of_flow	RF03	$\Sigma N_{rotina,j} == N_{bairro}$ para múltiplos instantes	PASS
test_exact_doc01_numerical_example	RF02	Vetor exato do Doc 01: [135.0, 105.0, 60.0, 0.0]	PASS
test_seasonal_modulation	RF04	Escalonamento linear para $\gamma = 1.5$ e $\gamma = 3.5$	PASS
test_circadian_periodicity	RF05	Invariância periódica $h(t) = h(t + 24k)$ para 24h e 168h	PASS
test_gate_weights_validation_error	RF03	Disparo de exceção para pesos com soma $\neq 1.0$	PASS
test_execution_performance	RNF01	Complexidade $O(1)$: 1.356 μs/ciclo (limite: 50 μ s)	PASS
test_generate_24h_curve	RNF02	96 pontos gerados com pico em 16:30	PASS
test_pydantic_response_serialization	RNF03	Serialização conforme contratos REST FastAPI	PASS

6. Curva Circadiana Amostral de 24 Horas

Amostras horárias do fluxo simulado geradas pelo motor matemático, demonstrando o piso da madrugada e a curva suave até o ápice turístico vespertino:

Horário	Hora (dec)	Volume (Pedestres)	Lacerda (45%)	Sé (35%)	Carmo (20%)	Pelourinho (0%)
00:00	0.00	15.0 pessoas	6.8	5.3	3.0	0.0
03:00 (Piso)	3.00	15.0 pessoas	6.8	5.3	3.0	0.0
08:00	8.00	20.1 pessoas	9.0	7.0	4.0	0.0
12:00	12.00	107.5 pessoas	48.4	37.6	21.5	0.0
14:00	14.00	216.4 pessoas	97.4	75.7	43.3	0.0
16:30 (Ápice)	16.50	300.0 pessoas	135.0	105.0	60.0	0.0
18:00	18.00	266.5 pessoas	119.9	93.3	53.3	0.0
21:00	21.00	107.5 pessoas	48.4	37.6	21.5	0.0

7. Conclusão & Próximos Passos

A implementação do **Doc 01** entrega o motor analítico circadiano de referência pronto para o barramento. Os passos subsequentes do roadmap de dados sintéticos são:

- Doc 02 (Injeção de Eventos):** Modelagem dos pulsos sigmoidais culturais de shows e ensaios do Olodum;
- Persistência GeoPackage / SQLite:** Injeção direta da série temporal simulada no Datalake compartilhado (/infra/dados) para recarga a quente no QGIS Server e visualização no Leaflet.